

HarmonyOS 开发者认证题库（高级）

实操

1. 无重复最长子串

给定一个字符串s，请你找出其中不含有重复字符的最长子串的长度(英文字母需要区分大小写)

解答要求

时间限制: 2000ms

内存限制: 2000MB

输入

输入一个字符串，字符串是由英文字母(包含大小写)和数字组成，长度不超过30.000。

输出

输出一个正整数代表不含有重复字符的最长子串的长度，

样例1

复制 输入:abcabcbb

复制 输出:3

解释:最长无重复字符的最长子串是“abc”。

样例2

复制 输入:pwwWew

复制 输出:3

解释:最长无重复字符的最长子串是“wWe”。

代码:

```
@Entry
@Component
struct UniqueSubStringLength {
  // 存储用户输入的字符串，@State装饰确保输入变化时UI自动更新
  @State inputStr: string = ""
  // 存储计算结果，用于在界面展示
  @State result: string = '请输入字符串并计算'

  // 计算无重复字符的最长子串长度
  private getMaxLength(s: string): number {
    // Set集合：存储当前窗口中出现的字符（自动去重）
    const charSet = new Set<string>();
    // maxLen：最大长度，记录遍历过程中找到的最长子串长度 left：左边界（初始在0位置）
    let maxLen = 0, left = 0;
    // 右边界从0开始遍历字符串
    for (let right = 0; right < s.length; right++) {
      // 当前右指针指向的字符
      const currentChar = s[right];
      // 若字符已存在，移动左指针并移除窗口外的字符
      while (charSet.has(currentChar)) {
```

```

    // 移除左指针指向的字符（从窗口中剔除），左指针右移，缩小窗口范围
    charSet.delete(s[left++]);
  }
  // 将当前字符加入窗口（此时窗口中无重复）
  charSet.add(currentChar);
  // 更新最大长度：当前窗口长度（right - left + 1）与历史最大值比较
  maxLen = Math.max(maxLen, right - left + 1);
}
return maxLen;
}

```

// 处理计算逻辑：获取输入、调用算法、更新结果

```

private calculate() {
  // 去除输入前后空格，避免空输入或纯空格输入
  const str = this.inputStr.trim();
  if (str === "") {
    this.result = '输入不能为空，请重新输入';
    return;
  }
  // 调用核心算法计算结果,更新结果文本
  const length = this.getMaxLength(str);
  this.result = `最长无重复子串长度: ${length}`;
}

```

```

build() {
  Column() {
    Text('无重复最长子串计算')
      .fontSize(20)
      .fontWeight(FontWeight.Bold)
      .margin({ bottom: 20 })
    TextInput({
      placeholder: '请输入字符串（包含大小写和数字）'
    })
      .width('80%')
      .height(40)
      .border({ width: 1 })
      .margin({ bottom: 20 }) // 输入变化时实时更新inputStr变量
      .onChange((value) => this.inputStr = value)
    Button('计算最长长度')
      .width('60%')
      .height(40)
      .onClick(() => this.calculate()) // 点击触发计算
    Text(this.result)
      .fontSize(15)
      .margin({ top: 15 })
  }
  .width('100%')
  .height('100%')
  .justifyContent(FlexAlign.Center)
}
}

```

2. “相关月”

在一年中月份的第一天星期数相同的两个月份。现在给你一年中的两个月份，判断这两个月份是否“相关”。

解答要求

时间限制: 2000ms

内存限制: 2000MB

输入一行字符串n(包含三个整数，依次为一个年份和两个月份)。

输出

如果相关就输出YES，不相关就输出NO。

输出

如果相关就输出YES，不相关就输出NO.

样例1

复制 输入:194814

复制 输出:YES

解释:这一年1月和4月的星期数相同。

样例2

复制 输入:1994109

复制 输出:NO

解释:这一年10月和9月的星期数不同

解析:

代码:

```
@Entry
@Component
struct RelatedMonths {
  @State inputText: string = "";
  @State result: string = "";
  @State showResult: boolean = false;

  // 计算指定年份和月份的第一天是星期几
  private getWeekday(year: number, month: number): number {
    // 使用Date构造函数构建
    const date = new Date(year, month - 1, 1);
    // getDay() 直接返回 0-6 (0=星期日, 6=星期六)
    return date.getDay();
  }

  // 判断两个月份是否相关
  private checkRelatedMonths(): void {
    this.showResult = true; // 默认设置显示结果
    const input = this.inputText.trim();
    // 验证输入长度
    if (input.length < 5) {
      this.result = '输入格式错误';
      return;
    }
    // 解析输入: 前4位年份, 剩余部分拆分月份
    const year = parseInt(input.substring(0, 4));
    const month1 = parseInt(input.substring(4, input.length - 1));
    const month2 = parseInt(input.substring(input.length - 1));
    // 验证月份范围并计算结果
    if (month1 < 1 || month1 > 12 || month2 < 1 || month2 > 12) {
```

```

        this.result = '月份必须在1-12之间';
    } else {
        // 直接通过星期数是否相等输出结果
        this.result = this.getWeekday(year, month1) === this.getWeekday(year, month2)
? 'YES' : 'NO';
    }
}

build() {
    Column() {
        Text('相关月')
            .fontSize(20)
            .fontWeight(FontWeight.Bold)
        TextInput({ placeholder: '输入年份和月份(如194814)' })
            .width('80%')
            .height(40)
            .border({ width: 1 })
            .onChange((value: string) => {
                this.inputText = value;
                this.showResult = false;
            })
        Button('判断是否相关')
            .width('60%')
            .height(40)
            .onClick(() => {
                this.checkRelatedMonths();
            })
        //显示结果
        if (this.showResult) {
            Text(this.result)
                .fontSize(20)
                .fontWeight(FontWeight.Bold)
                .fontColor(this.result === 'YES' ? Color.Green : Color.Red)
        }
    }
    .width('100%')
    .height('100%')
    .justifyContent(FlexAlign.Center)
}
}

```

3. 四：实现：“奇”为奇，“偶”为偶

小华在做鸿蒙开发编程练习题时发现，有些整数按从低位到高位顺序(个、十、百、千.....个位是第一个奇数位)，奇数位上的数字总是奇数，偶数位上的数字总是偶数，例如21，奇数位(个位)为1;偶数位(十位)为2。给定一个正整数N，请你计算从1到N一共有多少个这种数。

代码：

```

@Entry
@Component
struct OddEvenCounter {
    @State inputValue: string = "

```

```

@State result: number = 0
@State tip: string = '请输入正整数N'

/** 判断数字是否符合条件 */
private isValid(num: number): boolean {
    const s = num.toString()
    for (let i = 0; i < s.length; i++) {
        const pos = i + 1 // 位序 (1开始)
        const digit = parseInt(s[s.length - 1 - i]) // 从个位取数
        // 奇数位需为奇, 偶数位需为偶
        if ((pos % 2 === 1 && digit % 2 === 0) || (pos % 2 === 0 && digit % 2 === 1)) {
            return false
        }
    }
    return true
}

/** 计算1到N中符合条件的数字总数 */
private calculate(N: number): number {
    let count = 0
    for (let i = 1; i <= N; i++) {
        if (this.isValid(i)) {
            count++
        }
    }
    return count
}

// 验证处理计算逻辑
private handleCalc() {
    const N = parseInt(this.inputValue.trim())
    if (isNaN(N) || N <= 0) {
        this.tip = '请输入有效的正整数'
        this.result = 0
    } else {
        this.tip = `1到${N}中符合条件的数字个数: `
        this.result = this.calculate(N)
    }
}

build() {
    Column() {
        Text('奇为奇, 偶为偶计算器')
            .width('80%')
            .fontSize(20)
            .fontWeight(FontWeight.Bold)
            .margin({ bottom: 20 })
        TextInput({ placeholder: '例如: 100' })
            .width('80%')
            .height(40)
            .border({ width: 1 })
            .margin({ bottom: 20 })
            .onChange((v) => this.inputValue = v)
        Button('计算结果')
            .width('60%')
    }
}

```

```

.height(40)
.margin({ bottom: 20 })
.onClick(() => this.handleCalc())
Text(this.tip).fontSize(15)
.margin({ bottom: 20 })
Text(this.result.toString())
.fontSize(15)
.fontWeight(FontWeight.Bold)
.fontColor(Color.Red)
}
.width('100%')
.height('100%')
.justifyContent(FlexAlign.Center)
}
}

```

4. 删除回文子序列（参考模版）

1. 删除回文子序列

给你一个字符串 s ，它仅由字母 'a' 和 'b' 组成。每一次删除操作都可以从 s 中删除 1 个回文子序列。

返回删除给定字符串中所有字符（字符串为空）的最小删除次数。

「子序列」定义：如果一个字符串可以通过删除原字符串某些字符而不改变原字符串顺序得到，那么这个字符串就是原字符串的一个子序列。

「回文」定义：如果一个字符串向后和向前读是一致的，那么这个字符串就是一个回文。

解答要求

时间限制: 2000ms
内存限制: 2000MB

输入

输入只有一行，包含一个字符串 s ，仅包含字母 'a' 和 'b'。
其中 $1 \leq s.length \leq 1000$

输出

在一行输出一个整数，返回删除给定字符串中所有字符（字符串为空）的最小删除次数。

样例1

复制 输入: abb
复制 输出: 2
解释: "abb" -> "bb" -> "".

```

1
2 process.stdin.resume();
3 process.stdin.setEncoding('utf-8');
4 let input = '';
5 process.stdin.on('data', (data) => {
6   input += data;
7 });
8 process.stdin.on('end', () => {
9   let inputArray = input.split('\n');
10  // 待实现函数，在此函数中填入解题代码
11  // doFunc()
12  // 输出结果
13
14  function doFunc() {
15    // 取输入字符串（去除首尾空白字符）
16    const s = 'ababa';
17    // 双指针判断是否为回文
18    let left = 0;
19    let right = s.length - 1;
20    let isPal = true;
21    while (left < right) {
22      if (s.charAt(left) !== s.charAt(right)) {
23        isPal = false;
24        break;
25      }
26      left++;
27      right--;
28    }
29    // 输出结果：回文则1次，否则2次；空字符串0次
30    const result = s.length === 0 ? 0 : (isPal ? 1 : 2);
31    console.log(result);
32  }
33  doFunc();

```

给你一个字符串 s ，它仅由字母 'a' 和 'b' 组成。每一次删除操作都可以从 s 中删除 1 个回文子序列。

返回删除给定字符串中所有字符（字符串为空）的最小删除次数。

「子序列」定义：如果一个字符串可以通过删除原字符串某些字符而不改变原字符串顺序得到，那么这个字符串就是原字符串的一个子序列。

「回文」定义：如果一个字符串向后和向前读是一致的，那么这个字符串就是一个回文。

解答要求

时间限制: 2000ms

内存限制: 2000MB

输入

输入只有一行，包含一个字符串 s ，仅包含字母 'a' 和 'b'。其中 $1 \leq s.length \leq 1000$

输出

在一行输出一个整数，返回删除给定字符串中所有字符（字符串为空）的最小删除次数。

样例1

复制 输入: abb

复制 输出: 2

解释: "abb" -> "bb" -> "",

样例2

复制 输入: baabb

复制 输出: 2

解释: "baabb" -> "b" -> ""。

先删除回文子序列"baab", 然后再删除"b"。

样例3

复制 输入: ababa

复制 输出: 1

解释: 字符串本身就是回文序列。只需要删除一次。

代码:

```
process.stdin.resume();
process.stdin.setEncoding('utf-8');
let input = "";
process.stdin.on('data', (data) => {
    input += data;
});
process.stdin.on('end', () => {
    let inputArray = input.split('\n');
    /**
     * 待实现函数，在此函数中填入答题代码
     * doFunc()
     */
    function doFunc() {
        // 取输入的字符串（去除首尾空白字符）
        const s = inputArray[0].trim();
        // 双指针判断是否为回文
        let left:number = 0;
        let right:number = s.length - 1;
        let isPal = true;
        while (left < right) {
            if (s.charAt(left) !== s.charAt(right)) {
                isPal = false;
                break;
            }
            left++;
            right--;
        }
        // 输出结果：回文则1次，否则2次；空字符串则0次
        const result = s.length === 0 ? 0 : (isPal ? 1 : 2);
        console.log(result);
    }
    doFunc();
    process.exit();
});
```

5. 寻找相同子串

给定两个字符串t和p，要求从t中找到一个和p相同的连续子串，并输出p子串第一个字符在t中的下标。

解答要求

时间限制：2000ms

内存限制：2000MB

输入

输入文件包括两行，分别表示字符串t和p，保证t的长度不小于p，且t的长度不超过1000，p的长度不超过1000。

输出

如果能从t中找到一个和p相等的连续子串，则输出p子串第一个字符在t中的下标（下标从左到右依次为1,2,3..）；

如果不能则输出"No"；如果含有多个这样的子串，则输出第一个字符下标最小

代码：

编程题 第1/1题

1、寻找相同子串

给定两个字符串 t 和 p，要求从 t 中找到一个和 p 相同的连续子串，并输出 p 子串第一个字符在 t 中的下标。

解答要求

时间限制: 2000ms
内存限制: 2000MB

输入

输入文件包括两行，分别表示字符串 t 和 p，保证 t 的长度不小于 p，且 t 的长度不超过1000，p 的长度不超过1000。

输出

如果能从 t 中找到一个和 p 相等的连续子串，则输出 p 子串第一个字符在 t 中的下标（下标从左到右依次为1,2,3,...）；如果不能则输出"No"；如果含有多个这样的子串，则输出第一个字符下标最小

ArkTS

重置代码

```
1
2 process.stdin.resume();
3 process.stdin.setEncoding('utf-8');
4 let input = '';
5 process.stdin.on('data', (data) => {
6   input += data;
7 });
8 process.stdin.on('end', () => {
9   let inputArray = input.split('\n');
10  /**
11   * 待实现函数，在此函数中填入答题代码
12   * doFunc()
13   */
14   process.exit();
15 });
16
```

```
var readline = require('readline');
var rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});
```

```
var input = [];
rl.on('line', function(line) {
  input.push(line);
  if (input.length === 2) {
    var t = input[0];
    var p = input[1];
    var result = doFunc(t, p);
    console.log(result);
    rl.close();
  }
});
```

```
function doFunc(t, p) {
  var tLen = t.length;
  var pLen = p.length;

  for (var i = 0; i <= tLen - pLen; i++) {
    var match = true;
    for (var j = 0; j < pLen; j++) {
```



```

        if (t.charAt(i + j) !== p.charAt(j)) {
            match = false;
            break;
        }
    }
    if (match) {
        return i + 1; // 题目要求下标从1开始
    }
}
return "No";
}

```

6. 尼科彻斯定理

任何一个整数 m 的立方都可以写成多个连续奇数之和，给你一个整数输出和为整数的立方的连续奇数。

解答要求时间限制：2000ms内存限制：2000MB

输入

输入一行字符串 n （包含一个整数，大小为0-1000）

输出

找到和为 n^3 的连续奇数，输出所有连续奇数，以空格隔开。

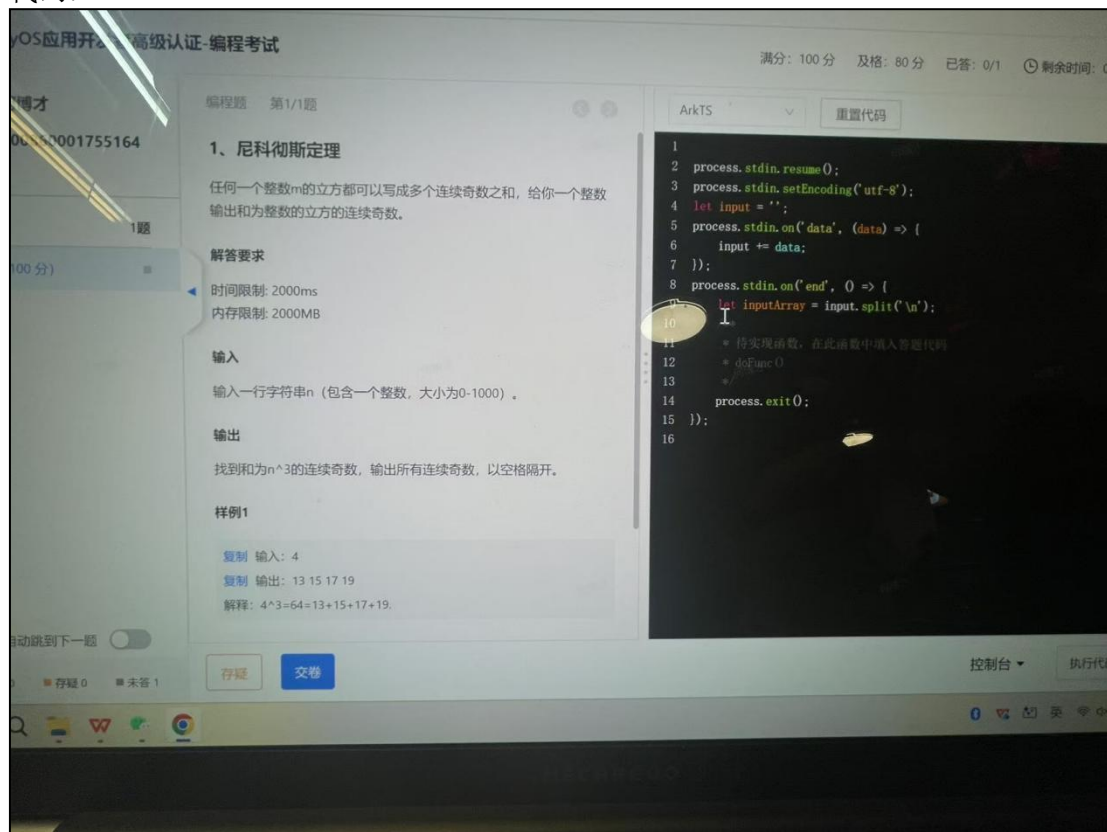
样例1

复制输入：4

复制输出：13151719

解释：4~3=64=13+15+17+19.

代码：



```

function doFunc(n) {
    var cube = n * n * n;

```

```

var start;

// 计算起始奇数
if (n % 2 === 1) {
    start = n * n - (n - 1);
} else {
    start = (n * n - 1) - (n - 2);
}

// 生成连续奇数序列
var result = [];
for (var i = 0; i < n; i++) {
    result.push(start + i * 2);
}

return result.join(' ');
}

// 从标准输入读取数据并处理
var readline = require('readline');
var rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout
});

rl.on('line', function(input) {
    var n = parseInt(input.trim(), 10);
    console.log(doFunc(n));
    rl.close();
});

```

7. 最大三角形面积

编程题 第1/1题

1、最大三角形面积

给你一个由 X-Y 平面上的点组成的数组 points，其中 points[i] = [xi, yi]。从其中取任意三个不同的点（其中一个点包含坐标轴原点）组成三角形，返回能组成的最大三角形的面积。与真实值误差在 10⁻⁵ 内的答案将会视为正确答案。

解答要求

时间限制: 2000ms
内存限制: 2000MB

输入

输入只有一行，包括一个字符串，各个二维点以逗号分隔。

输出

在一行输出一个字符串，返回以字符串形式地两个字符串的和，保留两位有效小数。

ArkTS

重置代码

```

1 process.stdin.resume();
2 process.stdin.setEncoding('utf-8');
3 let input = '';
4 process.stdin.on('data', (data) => {
5     input += data;
6 });
7 process.stdin.on('end', () => {
8     let inputArray = input.split('\n');
9     /**
10      * 待实现函数，在此函数中填入答题代码
11      * doFunc()
12      */
13     process.exit(0);
14 });
15
16

```

给你一个由X-Y平面上的点组成的数组points，其中points[i]=[xiyi]。从其中取任意三个不同的点（其中一个点包含坐标轴原点）组成三角形，返回能组成的最大三角形的面积。与真实值误差在10⁻⁵内的答案将会视为正确答案。

解答要求

时间限制：2000ms

内存限制：2000MB

输入

输入只有一行，包括一个字符串，各个二维点以逗号分隔。

输出

在一行输出一个字符串，返回以字符串形式地两个字符串的和，保留两位有效小数。

代码：

```
process.stdin.resume();
process.stdin.setEncoding('utf-8');
let input = "";
process.stdin.on('data', (data) => {
    input += data;
});

process.stdin.on('end', () => {
    // 修正1：正确解析逗号分隔的坐标
    const points = input.trim().split(',').map(item => {
        const [x, y] = item.trim().split(' ').map(Number);
        return [x, y];
    });

    // 修正2：实现面积计算函数
    function maxTriangleArea(points) {
        let maxArea = 0;
        const origin = [0, 0];

        // 确保包含原点的三角形
        const otherPoints = points.filter(p => !(p[0] === 0 && p[1] === 0));

        for (let i = 0; i < otherPoints.length; i++) {
            for (let j = i+1; j < otherPoints.length; j++) {
                const [x1, y1] = otherPoints[i];
                const [x2, y2] = otherPoints[j];
                // 向量叉积公式计算面积
                const area = 0.5 * Math.abs(x1*y2 - x2*y1);
                maxArea = Math.max(maxArea, area);
            }
        }
        return maxArea.toFixed(2);
    }

    console.log(maxTriangleArea(points));
    process.exit();
});
```

8. 连续整数之和

一个正整数有可能可以被表示为 $m(m>1)$ 个连续正整数之和，如： $15=1+2+3+4+5$

$15=4+5+6$

$15=7+8$

但现在你的任务是判断给定的整数 n 能否表示成连续的 $m(m>1)$ 个正整数之和

解答要求

时间限制：2000ms

内存限制：2000MB

输入

输入只有一个整数 n ($1 < n < 230 + 1$), 输出若 n 能表示成连续的 m ($m > 1$)个正整数之和则输出YES°, 否则输出"NO".

样例1

复制输入: 15

...

代码:

编程题 第1/1题

1、连续整数之和

一个正整数有可能可以被表示为 $m(m > 1)$ 个连续正整数之和, 如:

$15 = 1 + 2 + 3 + 4 + 5$
 $15 = 4 + 5 + 6$
 $15 = 7 + 8$

但现在你的任务是判断给定的整数 n 能否表示成连续的 $m(m > 1)$ 个正整数之和。

解答要求

时间限制: 2000ms
内存限制: 2000MB

输入

输入只有一个整数 n ($1 < n < 230 + 1$).

输出

若 n 能表示成连续的 $m(m > 1)$ 个正整数之和则输出"YES", 否则输出"NO".

样例1

复制 输入: 15

ArKTS

重置代码

```
1 process.stdin.resume();
2 process.stdin.setEncoding('utf-8');
3 let input = '';
4 process.stdin.on('data', (data) => {
5   input += data;
6 });
7 process.stdin.on('end', () => {
8   let inputArray = input.split('\n');
9   /**
10    * 待实现函数, 在此函数中填入答题代码
11    * doFunc()
12    */
13   process.exit();
14 });
15
16
```

答案:

```
process.stdin.resume();
process.stdin.setEncoding('utf-8');
let input = '';
process.stdin.on('data', (data) => {
  input += data;
});
process.stdin.on('end', () => {
  let inputArray = input.split('\n');
  let n = parseInt(inputArray[0].trim(), 10);

  function doFunc() {
    for (let m = 2; m * (m - 1) / 2 < n; m++) {
      if ((n - m * (m - 1) / 2) % m === 0) {
        console.log("YES");
        return;
      }
    }
    console.log("NO");
  }

  doFunc();
  process.exit();
});
```

9. 重复字母连续出现次数

一个字符串，只包含大写字母，求同一字母连续出现的最大次数，例如AAAABBCDHHH，同一字母连续出现的最大次数为4，因为一开始A连续出现了4次

解答要求

时间限制：2000ms

内存限制：2000MB

输入

输入一行字符串（1<长度<=100）

输出

输出同一个字母连续出现的最大次数

1、重复字母连续出现次数

一个字符串，只包含大写字母，求同一字母连续出现的最大次数。例如"AAAABBCDHHH"，同一字母连续出现的最大次数为4，因为一开始A连续出现了4次。

解答要求

时间限制：2000ms
内存限制：2000MB

输入

输入一行字符串(1<长度<=100)

输出

输出同一个字母连续出现的最大次数

```
1
2 process.stdin.resume();
3 process.stdin.setEncoding('utf-8');
4 let input = '';
5 process.stdin.on('data', (data) => {
6     input += data;
7 });
8 process.stdin.on('end', () => {
9     let inputArray = input.split('\n');
10    /**
11     * 待实现函数，在此函数中填入答题代码
12     * doFunc()
13     */
14    process.exit();
15 });
16
```

代码：

```
process.stdin.resume();
process.stdin.setEncoding('utf-8');
let input = '';
process.stdin.on('data', (data) => {
    input += data;
});
process.stdin.on('end', () => {
    let inputArray = input.split('\n');
    let str = inputArray[0].trim();

    function doFunc() {
        let maxCount = 0;
        let currentCount = 1;

        for (let i = 1; i < str.length; i++) {
            if (str[i] === str[i - 1]) {
                currentCount++;
            } else {
                if (currentCount > maxCount) {
                    maxCount = currentCount;
                }
                currentCount = 1;
            }
        }

        // 最后一段连续字符的处理
        if (currentCount > maxCount) {
```

```

        maxCount = currentCount;
    }

    console.log(maxCount);
}

doFunc();
process.exit();
});

```

10. 计算A+B

计算A+B，输出结果

解答要求

时间限制：2000ms

内存限制：2000MB

输入

输入两个参数A和B

输出

输出A和B的求和结果

样例1

复制 输入：1 1

2 3

复制 输出：2

5

解析：

编程题 第1/1题

1. 计算A+B

计算A+B，输出结果

解答要求

时间限制: 2000ms

内存限制: 2000MB

输入

输入两个整数A和B

输出

输出A和B的求和结果

样例1

复制 输入：1 1
2 3

复制 输出：2
5

解析：

样例2

复制 输入：2 8
3 9

ArkTS

查看代码

```

1
2 process.stdin.resume();
3 process.stdin.setEncoding('utf-8');
4 let input = '';
5 process.stdin.on('data', (data) => {
6     input += data;
7 });
8 process.stdin.on('end', () => {
9     let inputArray = input.split(' ');
10    /**
11     * 待实现函数，在此函数中填入答题代码
12     * doFunc()
13     */
14    process.exit(0);
15 });
16

```

代码：

```

process.stdin.resume();
process.stdin.setEncoding('utf-8');
let input = '';
process.stdin.on('data', (data) => {
    input += data;

```

```

});
process.stdin.on('end', () => {
  let inputArray = input.trim().split('\n');

  function doFunc() {
    for (let i = 0; i < inputArray.length; i++) {
      let nums = inputArray[i].split(' ').map(Number);
      if (nums.length === 2) {
        let a = nums[0];
        let b = nums[1];
        let sum = a + b;
        console.log(a);
        console.log(b);
        console.log(sum);
      }
    }
  }

  doFunc();
  process.exit();
});

```

11. 计算英文句子的单词重量

每个句子由多个单词组成，句子中的每个单词的长度都可能不一样，我们限设每个单词的长度 N 为该单词的重量，你需要做的就是找出整个句子的平均重量

解答要求

时间限制: 2000ms 内存限制 2000MB

输入

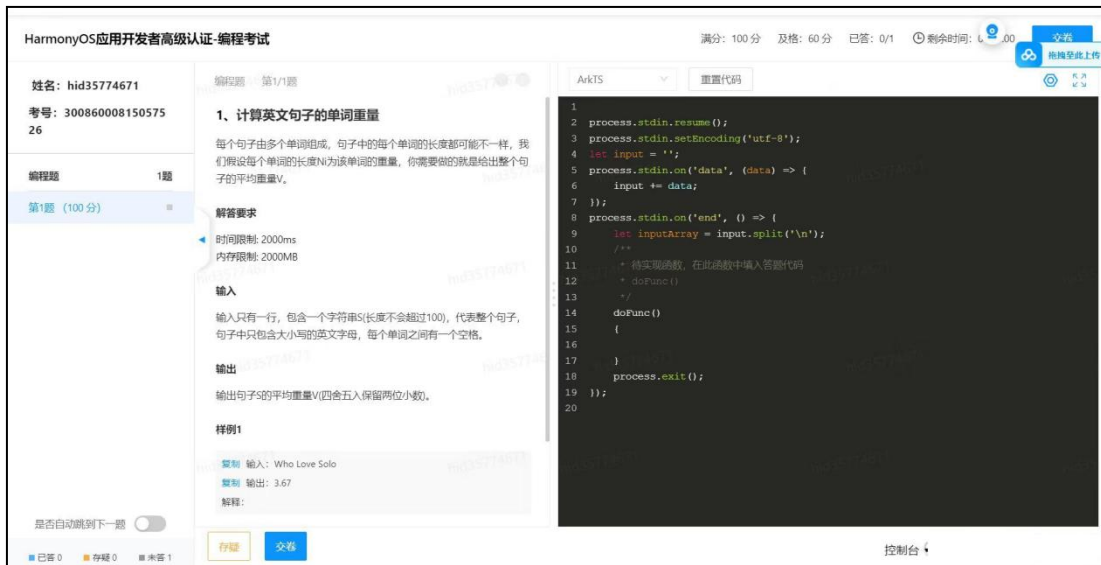
输入只有一行，包含一个字符串 S （长度不会超过100），代表整个句子，句子中只包含大小写的英文字母，每个单词之间有一个空路

输出

输出句子 S 的平均重量 V （四舍五入保留两位小数）

样例1复制输入: ho Love Solo

复制 输出: 3.67



代码

```
process.stdin.resume();
process.stdin.setEncoding('utf-8');
let input = '';
process.stdin.on('data', (data) => {
    input += data;
});
process.stdin.on('end', () => {
    let inputArray = input.split('\n');

    function doFunc() {
        let sentence = inputArray[0].trim();
        let words = sentence.split(' ');
        let totalLength = 0;

        for (let word of words) {
            totalLength += word.length;
        }

        let averageLength = totalLength / words.length;
        console.log(averageLength.toFixed(2));
    }

    doFunc();
    process.exit();
});
```

12. 丢失的数

给定一个包含 $[0, n)$ 中 n 个数的数相nums, 找出 $[0, n)$ 这个范围内没有出现在数组重的数, 如果有两个及以下的数字没有出现, 则输出false, 无缺失则输出true。

解答要求

时间限制: 2000ms

内存限制：2000MB

输入

输入一个以空格分隔的数组

输出

输出缺失的数字或者false或者true

样例1

复制 输入：D12

复制 输出：true

解释：因为数组长度是3，所以a=3.所以所有的数字都在[0,3]内，完整数组应该是0 1 2所以无丢失数字返回true

开发者高级认证-编程考试

086063 编程题 第1/1题

1、丢失的数

给定一个包含 [0, n) 中 n 个数的数组 nums，找出 [0, n) 这个范围内没有出现在数组中的数。如果有两个及以下的数字没有出现，则输出 false，无缺失则输出 true。

解答要求

时间限制：2000ms
内存限制：2000MB

输入

输入一个以空格分隔的数组

输出

输出缺失的数字或者false或者true

样例1

复制 输入：0 1 2
复制 输出：true
解释：因为数组长度是3，所以 n = 3，所以所有的数字都在范围 [0,3) 内，完整数组应该是 0 1 2 所以无丢失数字返回true

样例2

复制 输入：0 0 0 2

```
1 process.stdin.resume();
2 process.stdin.setEncoding("utf8");
3 let input = "";
4 process.stdin.on("data", (data) => {
5   input += data;
6 });
7 process.stdin.on("end", () => {
8   let inputArr = input.split("\n");
9   /*
10    待实现函数，在此函数中填入答题代码
11    doFunc ()
12    */
13   function doFunc(): boolean | number {
14     let nums = inputArr[0].split(" ").map(Number);
15     const n = nums.length;
16     const set = new Set<number>();
17     for (const num of nums) {
18       set.add(num);
19     }
20     let count = 0;
21     let missingNum: number | null = null;
22     for (let i = 0; i < n; i++) {
23       if (!set.has(i)) {
24         count++;
25         missingNum = i;
26         if (count > 1) {
27           return false;
28         }
29       }
30     }
31     return count === 0 ? true : missingNum!;
32   }
33   const res = doFunc();
34   console.log(res);
35   process.exit();
36 });
```

答案：

```
process.stdin.resume();
process.stdin.setEncoding("utf8");
let input = "";
process.stdin.on("data", (data) => {
  input += data;
});
process.stdin.on("end", () => {
  let inputArr = input.split("\n");
  /*
   待实现函数，在此函数中填入答题代码
   doFunc ()
  */
  function doFunc(): boolean | number {
    let nums = inputArr[0].split(" ").map(Number);
    const n = nums.length;
    const set = new Set<number>();
    for (const num of nums) {
      set.add(num);
    }
    let count = 0;
    let missingNum: number | null = null;
    for (let i = 0; i < n; i++) {
      if (!set.has(i)) {
        count++;
```

```
        missingNum = i;
        if (count > 1) {
            return false;
        }
    }
}
return count === 0 ? true : missingNum!;
}
const res = doFunc();
console.log(res);
process.exit();
});
```

13. 字母大小写转换

功能:输入一行字符，将此字符串中的元音字母(a,e,i,o,u,A,E,I,O,U)转换成大写字母，辅音字母转换成小写。

解答要求

时间限制: 2000ms

内存限制: 200MB

输入

输入一个字符串(长度不超过100，只包含大小写的英文字符和空格)。

输出

输出翻译后的字符序列。

样例1

输入:

Who Love Solo

输出:

wHO IOVE sOLO

解释: Who为辅音，改为小写，o为元音，改为大写，Love同理。

样例2

输入:

My name is ARTS

输出:

mY nAME IS arts

解释: 名字首字母，应该转为小写，ARTS为元音，所以改为大写



答案:

```
process.stdin.resume();
process.stdin.setEncoding("utf8");
let input = "";
process.stdin.on("data", (data) => {
  input += data;
});
process.stdin.on("end", () => {
  let inputArr = input.split("\n");
  /*
```

待实现函数，在此函数中填入答题代码

doFunc ())

```
*/
```

3-查询/搜索插入位置

```
function doFunc(): string {
  let input = inputArray[0].trim()
  let result = "";
  let length = input.length
  for (let i = 0; i < length; i++) {
    if ("aeiouAEIOU".indexOf(input[i]) !== -1) {
      result += input[i].toUpperCase();
    } else {
      result += input[i].toLowerCase();
    }
  }
  return result;
}
const res = doFunc();
console.log(res);
process.exit();
});
```

14. 最长元音子串的长度

定义： 当一个字符串只有元音字母(aeiouAEIOU)组成,称为元音字符串。现给定一个字符串,请找出其中最长的元音字符串,并返回其长度,如果找不到, 则返回0。

子串： 字符串中任意连续的字符组成的子序列称为该字符串的子串

解答要求

时间限制: 2000ms

内存限制: 2000MB

输入

一个字符串,其长度范围: $0 < \text{length} \leq 65535$,字符串仅由字母a-z和A-Z组成。

输出

一个整数,表示最长的元音字符串的长度。

样例1

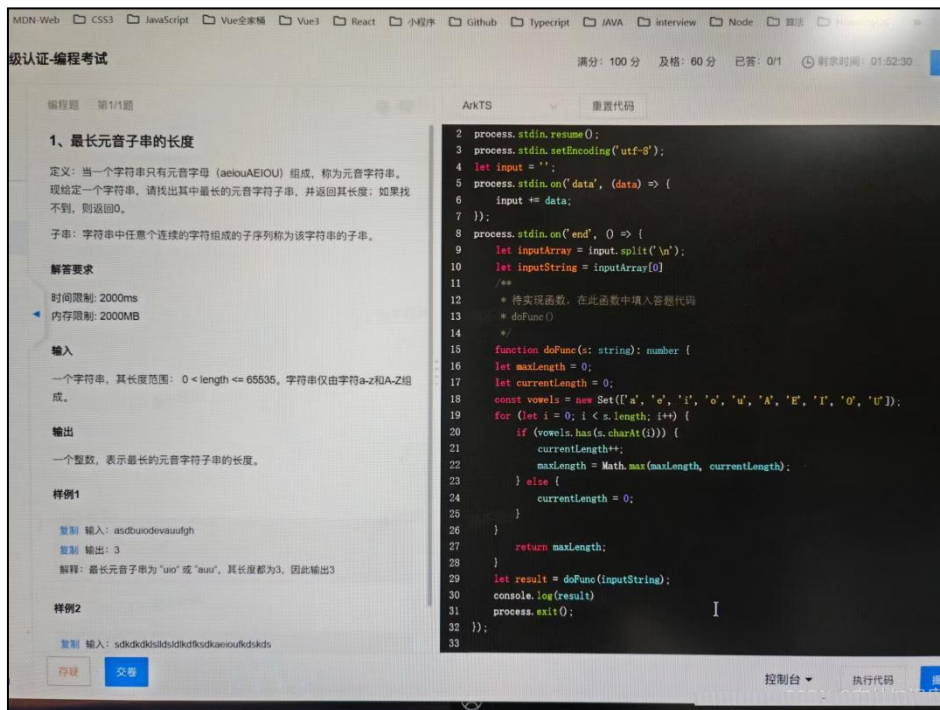
输入: asdfiuodevauugh

输出: 3

解释: 最长元音子串为 "uio" 或 "auu", 其长度都为3,因此输出3.

样例2

输入: abcdeffghijklmnopqrstuvwxyz



代码:

```
process.stdin.resume();
process.stdin.setEncoding("utf8");
let input = "";
process.stdin.on("data", (data) => {
  input += data;
});
process.stdin.on("end", () => {
  let inputArray = input.split("\n");
  let inputString = inputArray[0]
```

```

/*
待实现函数，在此函数中填入答题代码
doFunc ( ) )
*/
function doFunc(s:string): number {
  let maxLength = 0;
  let currentLength = 0;
  const vowels = new Set(['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U']);
  for (let i = 0; i < s.length; i++) {
    if (vowels.has(s.charAt(i))) {
      currentLength++;
      maxLength = Math.max(maxLength, currentLength);
    } else {
      currentLength = 0;
    }
  }
  return maxLength;
};
let result = doFunc(inputString);
console.log(result);
process.exit();
});

```

15. 查询插入位置

内存限制: 20000MB

输入

给定一个字符串,依次包含一个升序数组和一个目标值,每个数字用空格隔开

输出

如果在数组中找到目标值,返回其索引。如果目标值不存在于数组中,返回它将会被按顺序插入的位置。

样例1

复制 输入: 1 3 5 6

复制 输出: 2

解释: 数组为[1,3,5,6],目标值为5,数组内存在5,索引为2,输出2

样例2

复制 输入: 1 3 5 6 2

复制 输出: 1

解释: 数组为[1,3,5,6,2],目标值为2,数组内不存在2,按顺序插入要在索引1插入,输出1

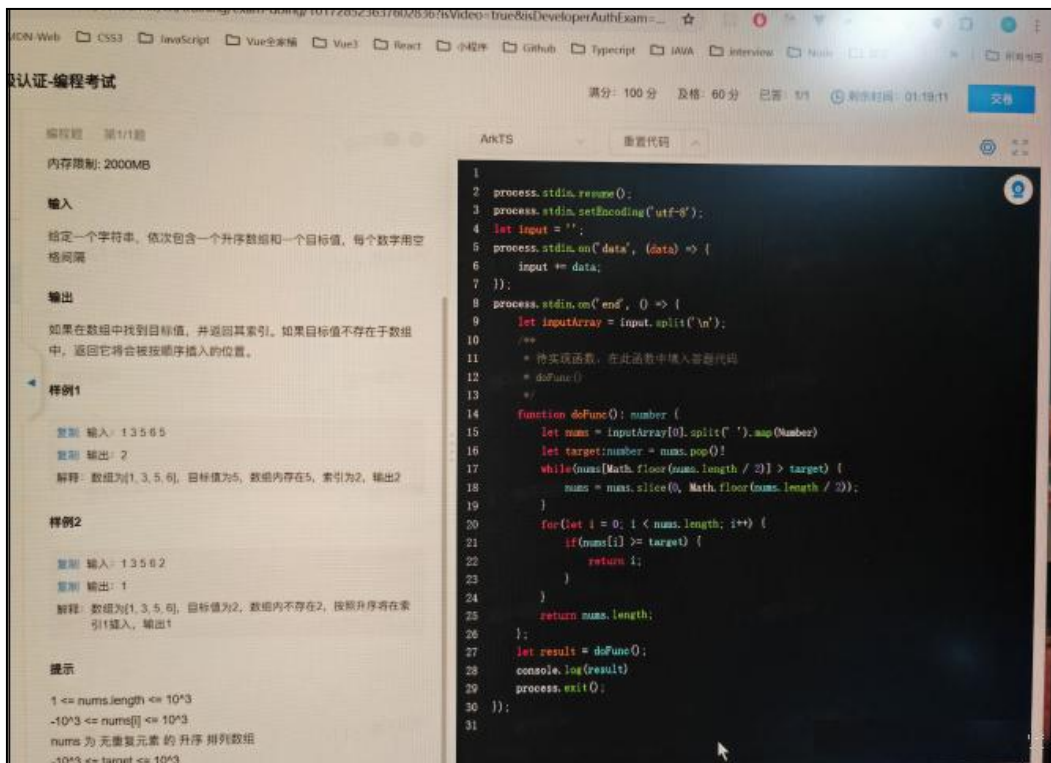
提示

$1 \leq \text{nums.length} \leq 10^3$

$-10^3 \leq \text{nums}[i] \leq 10^3$

nums 为无重复元素的 升序 排列数组

$-10^3 \leq \text{target} \leq 10^3$



代码:

```
process.stdin.resume();
process.stdin.setEncoding("utf8");
let input = "";
process.stdin.on("data", (data) => {
  input += data;
});
process.stdin.on("end", () => {
  let inputArr = input.split("\n");
  /*
   待实现函数，在此函数中填入答题代码
   doFunc ( )
  */
  function doFunc(): number {
    let nums = inputArray[0].split(' ').map(Number)
    let target:number = nums.pop()!
    while(nums[Math.floor(nums.length / 2)] > target) {
      nums = nums.slice(0, Math.floor(nums.length / 2));
    }
    for(let i = 0; i < nums.length; i++) {
      if(nums[i] >= target) {
        return i;
      }
    }
    return nums.length;
  };
  let result = doFunc();
  console.log(result);
  process.exit();
});
```

16. N整数阶乘

给定一个整数 $n(0 \leq n \leq 1000)$,您的任务是计算 n 的阶乘。

解答要求

时间限制: 2000MB

内存限制: 2000MB

输入

输入第一行为一个整数 $N(0 \leq N \leq 10000)$ 。(OCR原文为 $0 \leq N \leq 10000$)

输出

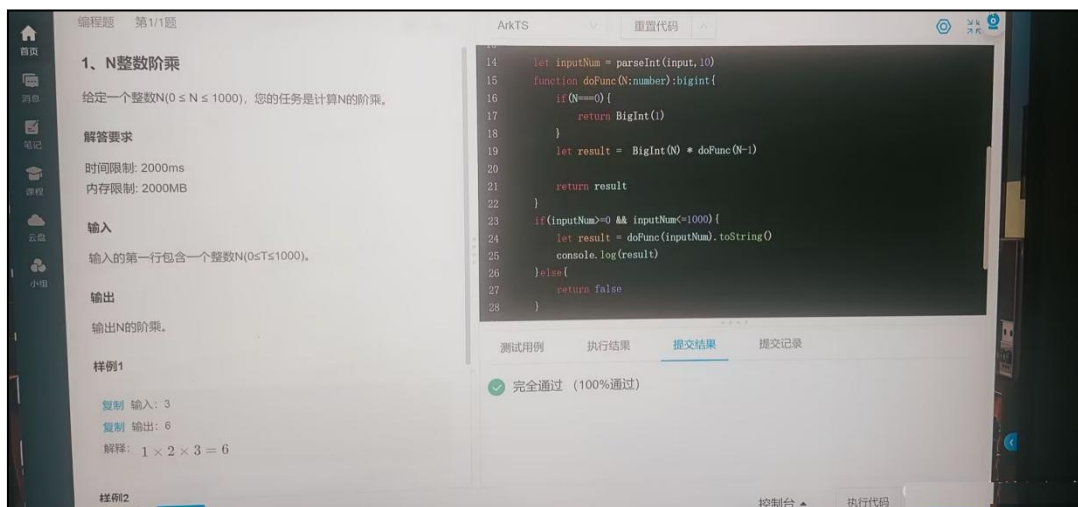
输出 n 的阶乘。

样例:

示例 输入: 5

输出: 6

解释: $1 \times 2 \times 3 = 6$



答案:

```
process.stdin.resume();
process.stdin.setEncoding("utf8");
let input = "";
process.stdin.on("data", (data) => {
    input += data;
});
process.stdin.on("end", () => {
    /*
    待实现函数，在此函数中填入答题代码
    doFunc ( )
    */
    let inputNum = parseInt(input, 10)
    function factorial(N: number): bigint {
        if (N === 0) {
            return BigInt(1);
        }
        return BigInt(N) * factorial(N - 1);
    }
    if (inputNum >= 0 && inputNum <= 1000) {
        const result = factorial(inputNum).toString();
        console.log(result);
    } else {
```

```
console.log('输入的整数不符合要求');
}
process.exit();
});
```

17. 最大公因数

功能:给定两个数,输出这两个数的最大公约数。

时间限制: 2000ms

内存限制: 200MB

输入

输入一个数字n长度不超过100,只包含两个要计算的整数和空格。

输出

输出只有一行,即按要求输出两数的最大公因数。

示例1

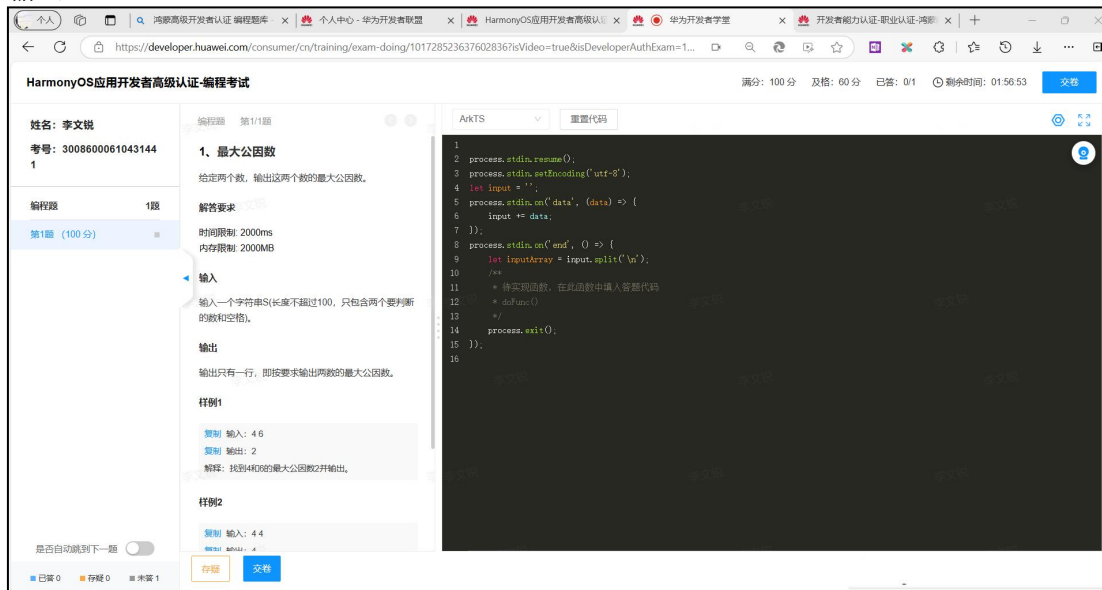
输入:4 6

输出:2

示例2

输入:4 4

输出:4



答案:

```
process.stdin.resume();
process.stdin.setEncoding("utf8");
let input = "";
process.stdin.on("data", (data) => {
  input += data;
});
process.stdin.on("end", () => {
  /*
   待实现函数, 在此函数中填入答题代码
   doFunc ()
  */
});
```



```
let inputArray = input.split("\n");
const a = Number(inputArray.toString().split(" ")[0])
const b= Number(inputArray.toString().split(" ")[1])
function doFunc(a: number, b: number): number {
  while (b!= 0) {
    let temp = b;
    b = a % b;
    a = temp;
  }
  return a;
}
let result = doFunc(a, b);
console.log(result);
process.exit();
});
```

18. 与后续更小值之差

给你一个数组**nums**，其中**nums[i]**是商店里第*i*件商品的价格。

商店里正在进行促销活动，如果你要买某件商品，那么你可以得到与后面首个比这件商品价格低的商品价格的折扣，如果没有满足条件的商品，你将没有任何折扣。

请你返回一个数组，数组中第*i*个元素的值是**nums[i]**与**nums[i]**后首个更小值的差值，如果没有更小值，则为**nums[i]**。

解答要求

时间限制: 2000ms

内存限制: 2000MB

输入

输入一个使用空格间隔的数字字符串**nums**。

输出

输出一个使用空格间隔的数字字符串，数组中第*i*个元素的值是**nums[i]**与**nums[i]**后首个更小值的差值，如果没有更小值，则为**nums[i]**。

样例1

复制输入: 8 4 6 2 3

复制输出: 4 2 4 2 3

解释:

nums[0]=8, 其后首个更小值为**nums**[1]=4, 所以差值为 $8-4 = 4$,

nums[1]=4, 其后首个更小值为**nums**[3]=2, 所以差值为 $4-2 = 2$,

nums[2]=6, 其后首个更小值为**nums**[3]=2, 所以差值为 $6-2 = 4$,

nums[3]=2, 其后无更小值, 所以差值为 $2-0 = 2$,

nums[4]=3, 其后无更小值, 所以差值为 $3-0 = 3$

1、与后续更小值之差

给你一个数组 `nums`，其中 `nums[i]` 是商店里第 i 件商品的价格。

商店里正在进行促销活动，如果你要买某件商品，那么你可以得到与后面首个比这件商品价格低的价格的折扣，如果没有满足条件的商品，你将没有任何折扣。

请你返回一个数组，数组中第 i 个元素是折扣后你购买商品 i 最终需要支付的价格。

给你一个数组 `nums`，请你返回一个数组，数组中第 i 个元素的值是 `nums[i]` 与 `nums[i]` 后面首个更小值的差值，如果没有更小值，则为 `nums[i]`。

解答要求

时间限制: 2000ms

内存限制: 2000MB

输入

输入一个使用空格间隔的数组字符串 `nums`。

输出

输出一个使用空格间隔的数组字符串，数组中第 i 个元素的值是 `nums[i]` 与 `nums[i]` 后面首个更小值的差值，如果没有更小值，则为 `nums[i]`。

样例1

复制 输入: 8 4 6 2 3

复制 输出: 4 2 4 2 3

解释: `nums[0]=8`，首个更小值为 `nums[1]=4`，所以差值为 $8-4=4$ 。
`nums[1]=4`，首个更小值为 `nums[3]=2`，所以差值为 $4-2=2$ 。
`nums[2]=6`，首个更小值为 `nums[3]=2`，所以差值为 $6-2=4$ 。
`nums[3]=2`，后面无更小值，所以差值为 $2-0=2$ 。
`nums[4]=3`，后面无更小值，所以差值为 $3-0=3$ 。

样例2

```

12  *  doFunc(): string {
13  */
14  function doFunc(): string {
15      // 将输入的字符串按空格分割为数字字符串数组，再转换为数字数组
16      let nums: number[] = inputArray[0].split(' ').map(Number);
17      //console.log(nums);
18
19      let result: number[] = [];
20      for (let i = 0; i < nums.length; i++) {
21          let foundSmaller = false;
22          for (let j = i + 1; j < nums.length; j++) {
23              if (nums[j] < nums[i]) {
24                  result.push(nums[i] - nums[j]);
25                  foundSmaller = true;
26                  break;
27              }
28          }
29          if (!foundSmaller) {
30              result.push(nums[i]);
31          }
32      }
33      // 将结果数字数组转换为使用空格间隔的字符串
34      return result.map(String).join(' ');
35  }
36
37  // 示例用法
38
39  let outputNumsStr = doFunc();
40  console.log(outputNumsStr);
41  process.exit();
42  }};
43

```

测试用例 执行结果 提交结果 提交记录

完全通过 (100%通过)

代码:

```

process.stdin.resume();
process.stdin.setEncoding("utf8");
let input = "";
process.stdin.on("data", (data) => {
    input += data;
});
process.stdin.on("end", () => {
    let inputArray = input.split("\n");
    /*
    待实现函数，在此函数中填入答题代码
    doFunc ( )
    */
    function doFunc():string{
        let nums: number[] = inputArray[0].split(' ').map(Number);
        let result: number[] = [];
        for (let i = 0; i < nums.length; i++) {
            let foundSmaller = false;
            for (let j = i + 1; j < nums.length; j++) {
                if (nums[j] < nums[i]) {
                    result.push(nums[i] - nums[j]);
                    foundSmaller = true;
                    break;
                }
            }
            if (!foundSmaller) {
                result.push(nums[i]);
            }
        }
        return result.map(String).join(' ');
    }
    let res = doFunc()
    console.log(res)
    process.exit();
});

```

19. 整数分解

给定一个整数 n ，将它拆分成它所有最小因子（大于1）的连乘形式，同时遵守最小因子优先的原则，小的因子在前面，输出的时候以空格间隔，如 $n=6$ ，则输出2 3。

解答要求

时间限制: 4000ms

内存限制: 2000MB

输入

测试包含一组数据，每个测试包含一个整数 n ($1 \leq n \leq 1000000$)，输入到文件末尾结束。

输出

每个测试输出一行，即分解的结果，所有因子按照从小到大排序输出，以空格间隔。

样例1

复制 输入: 5

复制 输出: 5



代码:

```
process.stdin.resume();
process.stdin.setEncoding('utf-8');
let input = '';
process.stdin.on('data', (data) => {
    input += data;
});
process.stdin.on('end', () => {
    let inputArray = input.split('\n');
    let c = parseInt(inputArray[0]);

    function integerFactorization(n: number): number[] {
        let factors: number[] = [];
        let i: number = 2;
        while (n > 1) {
            while (n % i === 0) {
                factors.push(i);
                n /= i;
            }
            i++;
        }
        return factors;
    }

    let factors = integerFactorization(c);
    console.log(factors.join(' '));
}
```

```

        i++;
    }
    return factors;
}
let result: number[] = integerFactorization(c);
console.log(result.join(" "));
process.exit();
});

```

20. 计算可用的号码数量

给定一个号码区间[start, end], 包含4或18的号码都不能使用, 如: 418、148、718不能使用, 108可用。请问有多少可用号码。

解答要求

时间限制: 2000ms

输入

两个整数start end(单空格间隔), 用于标识号码区间, $0 \leq \text{start} < \text{end} \leq 100000$

输出

一个整数, 代表可用号码的数量

样例1

复制 输入: 3 20

复制 输出: 15

解释: 号码3-20间为18, 不能使用的号码为4、14、18数量为3, 所以输出15

样例2

编程题 第1/1题

1、计算可用号码数量

给定一个号码区间[start, end], 包含4或18的号码都不能使用, 如: 418、148、718不能使用, 108可用。请问有多少可用号码。

解答要求

时间限制: 2000ms

输入

两个整数start end(单空格间隔), 用于标识号码区间, $0 < \text{start} < \text{end} \leq 100000$

输出

一个整数, 代表可用号码的数量

样例1

复制 输入: 3 20
复制 输出: 15
解释: 号码3-20总数为18, 不能使用的号码为4、14、18数量为3, 所以输出15

样例2

ArkTS

```

4 let input = process.stdin.read();
5 process.stdin.resume();
6 process.stdin.setEncoding('utf-8');
7 input = input.trim().split(' ');
8 let start = Number(input[0]);
9 let end = Number(input[1]);
10
11 /**
12  * 待实现函数, 在此函数中填入答题代码
13  * doFunc()
14  */
15 const start = Number(inputArray.toString().split(" ")[0]);
16 const end = Number(inputArray.toString().split(" ")[1]);
17 function doFunc(start: number, end: number): number {
18     let count = 0;
19     for (let num = start; num <= end; num++) {
20         let numStr = num.toString();
21         if (numStr.indexOf('4') === -1 && numStr.indexOf('18') === -1) {
22             count++;
23         }
24     }
25     return count;
26 }
27 let result = doFunc(start, end);
28 console.log(result);
29
30 process.exit(0);
31
32
33

```

控制台

执行代码

提交

代码:

```

process.stdin.resume();
process.stdin.setEncoding('utf-8');

```

```

let input = "";
process.stdin.on('data', (data) => {
    input += data;
});
process.stdin.on('end', () => {
    let inputArray = input.split("\n");
    /*
    待实现函数，在此函数中填入答题代码
    doFunc ( )
    */
    const start = Number(inputArray.toString().split(" ")[0])
    const end = Number(inputArray.toString().split(" ")[1])
    function doFunc(start: number, end: number): number {
        let count = 0;
        for (let num = start; num <= end; num++) {
            let numStr = num.toString();
            if (numStr.indexOf('4') === -1 && numStr.indexOf('18') === -1) {
                count++;
            }
        }
        return count;
    }
    let result = doFunc(start, end);
    console.log(result);
    process.exit();
});

```

21. 字符串相等

鸿蒙开发过程中,少不了对英文字母的识别,给定两个由大小写字母和空格组成的字符串s1和s2,判断压缩掉空格、并忽略大小写后,这两个字符串在是否相等。

解答要求

时间限制: 2000ms

内存限制: 2000MB

输入

输入两个字符串(分两行输入),只包含字母和空格,它们的长度都不超过100个字符。

输出

如果两个字符串相等则输出"Yes",否则输出"No"。

样例1

复制 输入: asdf aaa

aSdf aaa

复制 输出: Yes

解释:压缩空格,忽略大小写后就是相同的字符串 asdfaaa,所以输出Yes。

样例2

25

1、字符串相等

鸿蒙开发过程中，不少了对英文字母的识别，给定两个由大小写字母和空格组成的字符串s1和s2，判断压缩掉空格、并忽略大小写后，这两个字符串在是否相等。

解答要求

时间限制: 2000ms
内存限制: 2000MB

输入

输入两个字符串（分两行输入），只包含字母和空格，它们的长度都不超过 100 个字符。

输出

如果两个字符串相等则输出"Yes", 否则输出"No".

样例1

复制 输入: asdf aaa
aSdf aaa

复制 输出: Yes

解释: 压缩空格，忽略大小写后就是相同的字符串asdfaaa，所以输出Yes。

样例2

```

1 process.stdin.resume();
2 process.stdin.setEncoding('utf-8');
3 let input = '';
4 process.stdin.on('data', (data) => {
5     input += data;
6 });
7 process.stdin.on('end', () => {
8
9
10    let inputArray = input.split(/\r?\n/).filter(ele=>ele);
11    //console.log(inputArray)
12    let s1 = inputArray[0]
13    let s2 = inputArray[1]
14
15    /**
16     * 待实现函数，在此函数中填入答题代码
17     * doFunc()
18     */
19    function areEqualAfterProcess(s1:string,s2:string): string {
20        // 去除空格并转换为小写
21        let processedS1 = s1.replace(/\s/g, "").toLowerCase();
22        let processedS2 = s2.replace(/\s/g, "").toLowerCase();
23        return processedS1 === processedS2?"Yes":"No";
24    }
25
26    // 示例用法
27
28    let result = areEqualAfterProcess(s1, s2);
29    console.log(result);
30    process.exit();
31 });
32

```

代码:

```

process.stdin.resume();
process.stdin.setEncoding('utf-8');
let input = '';
process.stdin.on('data', (data) => {
    input += data;
});
process.stdin.on('end', () => {
    let inputArray = input.split("\n");
    /**
     待实现函数，在此函数中填入答题代码
     doFunc ( )
    */
    let s1 = inputArray[0]
    let s2 = inputArray[1]
    function areEqualAfterProcess(s1:string,s2:string): string {

        let processedS1 = s1.replace(/\s/g, "").toLowerCase();
        let processedS2 = s2.replace(/\s/g, "").toLowerCase();
        return processedS1 === processedS2?"Yes":"No";
    }
    let result = areEqualAfterProcess(s1, s2);
    console.log(result);
    process.exit();
});

```

22. 字符串大小写转换

输入一串字符，将把大写字母a~z,A~Z转换成大小写,辅助字母都写成小写。

屏幕要求

时间限制: 2000ms

内存限制: 2000KB

输入

输入一个字符串长度不超过150,只包括大小写英文字母和空格。

输出

输出转换后的字符序列。

样例1

输入: Who Love Solo

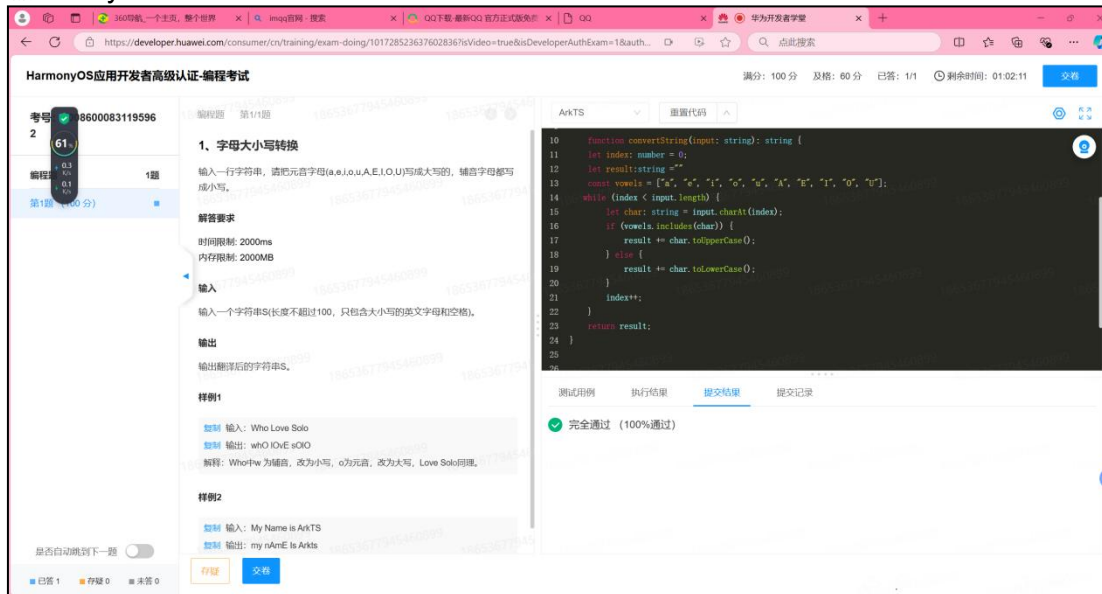
输出: who IOVE sOLO

解释: Who为辅格,改为小写, o为原图,改为大写, Love Solo同理。

样例2

输入: My Name is Aht78

输出: my nAME iS aht78



代码:

```
process.stdin.resume();
process.stdin.setEncoding('utf-8');
let input = "";
process.stdin.on('data', (data) => {
    input += data;
});
process.stdin.on('end', () => {
    //let inputArray = input.split("\n");
    /*
    待实现函数，在此函数中填入答题代码
    doFunc ( )
    */
    function convertString(input: string): string {
        let index:number = 0;
        let result:string = "";
```

```
const vowels = ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'];
while(index<input.length){
  let char:string = input.charAt(index)
  if(vowels.includes(char)){
    result += char.toUpperCase()
  }else{
    result += char.toLowerCase()
  }
  index++
}
return result
}
let result = convertString(input);
console.log(result);
process.exit();
});
```

23. 下一个更大值

给定一个用空格间隔的数组字符串num，返回一个数组字符串，其中每一项值为对于当前项的下一个更大值出现的序号间隔，如果在这之后不存在更大的值，请在该位置用0来代替。

解答要求

时间限制: 2000ms

内存限制: 2000MB

输入

输入一行用空格间隔的数组字符串。例如 70 71.

输出

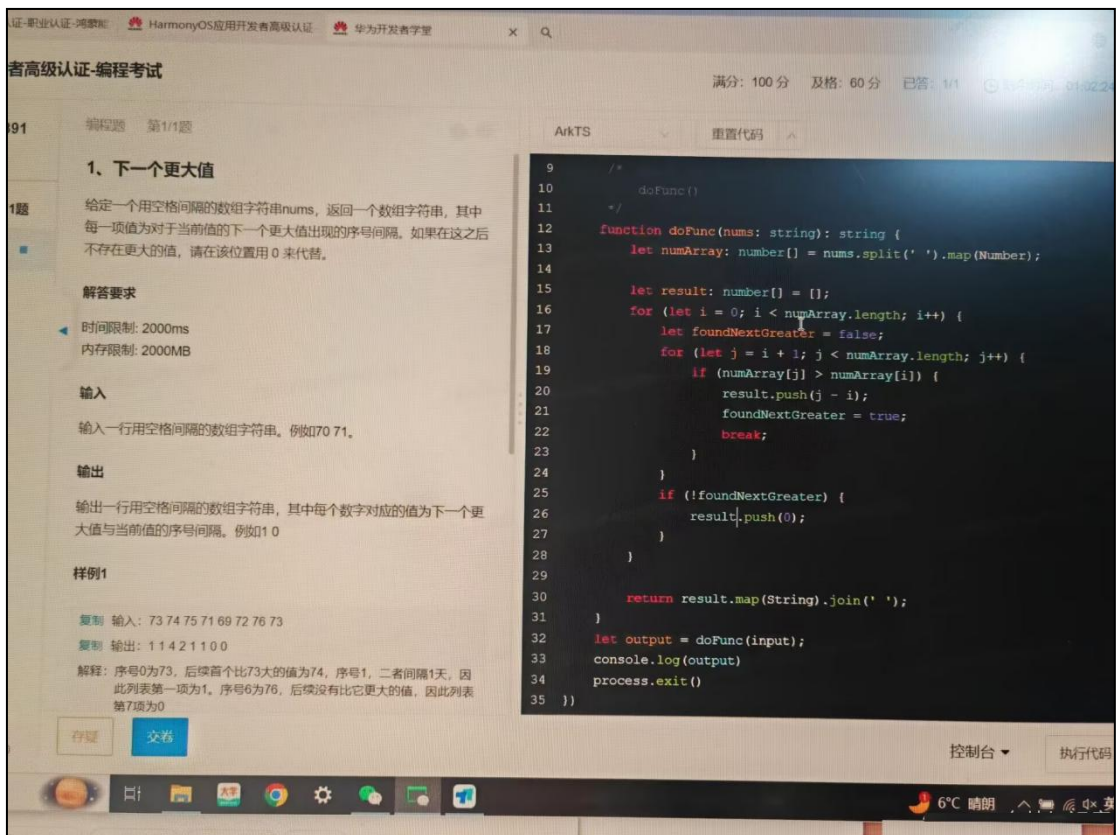
输出一行用空格间隔的数组字符串，其中每个数字对应的值为下一个更大值与当前值的序号间隔，例如1 0

样例1

输入: 73 74 75 71 69 72 76 73

输出: 1 1 4 2 1 1 0 0

说明：序号0为73，后面第一个比73大的值为74，序号1，二者间隔1天，因此列表第一位为1，序号6为73，后面没有比它更大的值，因此用0来代替。



代码:

```

process.stdin.resume();
process.stdin.setEncoding('utf-8');
let input = "";
process.stdin.on('data', (data) => {
    input += data;
});
process.stdin.on('end', () => {
    //let inputArray = input.split("\n");
    /*
    待实现函数，在此函数中填入答题代码
    doFunc ( )
    */
    function doFunc(nums: string): string {
        let numArray: number[] = nums.split(' ').map(Number);
        let result: number[] = [];
        for (let i = 0; i < numArray.length; i++) {
            let foundNextGreater = false;
            for (let j = i + 1; j < numArray.length; j++) {
                if (numArray[j] > numArray[i]) {
                    result.push(j - i);
                    foundNextGreater = true;
                    break;
                }
            }
            if (!foundNextGreater) {
                result.push(0);
            }
        }
        return result.map(String).join(' ');
    }
}

```

```
let output = doFunc(input);
console.log(output);
process.exit();
});
```

24. 超大数相加

在基于JVM的内存限制，无法加载超大数组

环境准备

初始容量: 200000m

内存容量: 400000m

输入

7个以上，8位大小不重复的数组，一个随机的起始位置，计算随机位置开始，往后累加8位不重复的数组的和
特殊情况的考虑边界值的兼容（起始位置为最大位99999999，起始位置开始没有8位不重复的数组）

输出

返回累加结果，大于int的最大值

步骤1

步骤 输入 值

数据 数组: 7,8,9,10,11,12,13

起始点 6

预期 77

结果 数组: 7,8,9,10,11,12,13

起始点 6

预期 77

结果 正确获得到77结果并返回结果

步骤2

步骤 输入 值

数据 数组: 1234567891011121314,8899101112131415,771011121314151617

起始点 99999999

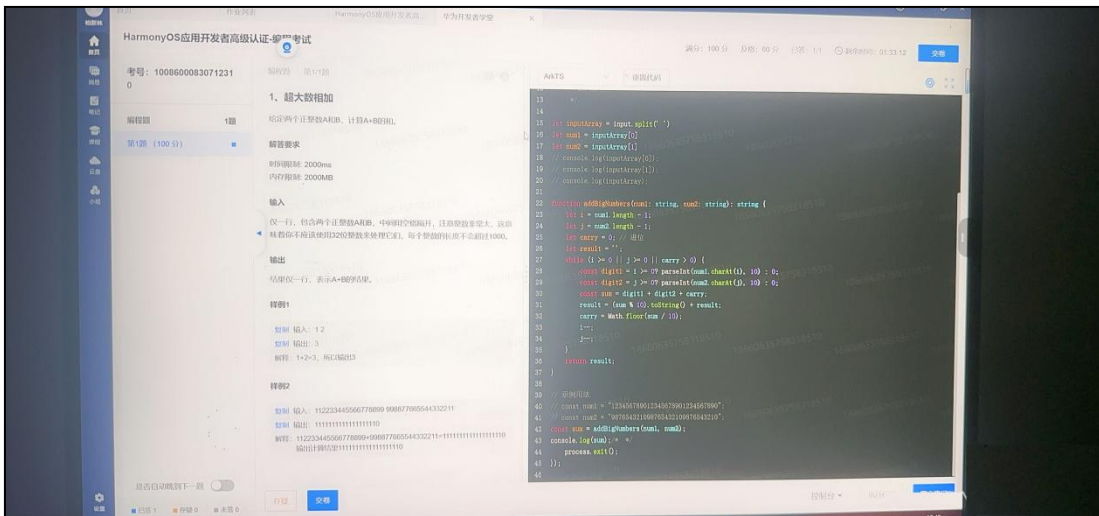
预期 报错

结果 数组: 1234567891011121314,8899101112131415,771011121314151617

起始点 99999999

预期 报错

结果 正确抛出异常并提示超出范围



代码:

```
process.stdin.resume();
process.stdin.setEncoding('utf-8');
let input = "";
process.stdin.on('data', (data) => {
    input += data;
});
process.stdin.on('end', () => {
    let inputArray = input.split("\n");
    /*
    待实现函数, 在此函数中填入答题代码
    doFunc ( )
    */
    let num1 = inputArray[0]
    let num2 = inputArray[1]
    function addBigNumbers(num1: string, num2: string): string {
        let i = num1.length - 1;
        let j = num2.length - 1;
        let carry = 0; // 进位
        let result = "";
        while (i >= 0 || j >= 0 || carry > 0) {
            const digit1 = i >= 0 ? parseInt(num1.charAt(i), 10) : 0;
            const digit2 = j >= 0 ? parseInt(num2.charAt(j), 10) : 0;
            const sum = digit1 + digit2 + carry;
            result = (sum % 10).toString() + result;
            carry = Math.floor(sum / 10);
            i--;
            j--;
        }
        return result;
    }
    let sum = addBigNumbers(num1, num2);
    console.log(sum);
    process.exit();
});
```

25. 平方数之和

给定一个非负整数 c ，请你判断是否存在两个不小于1的整数 a 和 b ，使得 $a^2 + b^2 = c^2$ 。

答题要求

时间限制：2000ms

内存限制：2000MB

输入

输入一个字符串 C （ C 的长度不超过100， C 只包含数字）。

输出

如果存在 $a^2 + b^2 = c^2$ ，输出 **TRUE**，不存在输出 **FALSE**。

样例1

输入：5

输出：TRUE

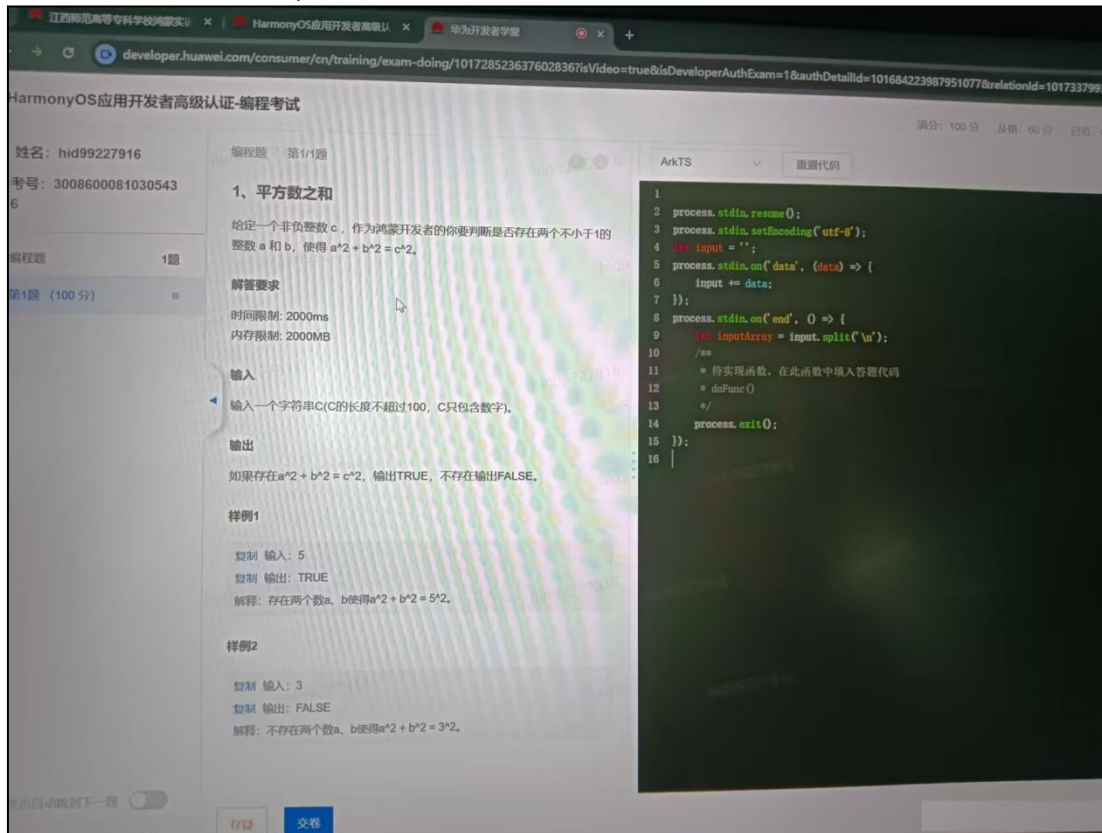
解释：存在两个数 $a=3, b=4$ ，使得 $3^2 + 4^2 = 5^2$ 。

样例2

输入：3

输出：FALSE

解释：不存在两个数 a, b ，使得 $a^2 + b^2 = 3^2$ 。



代码:

```
process.stdin.resume();
process.stdin.setEncoding('utf-8');
let input = '';
process.stdin.on('data', (data) => {
  input += data;
});
process.stdin.on('end', () => {
  //let inputArray = input.split("\n");
```

```

/*
待实现函数，在此函数中填入答题代码
doFunc ( )
*/
let inputArray = input.split("\n")
let n = Number(inputArray[0].split(""))
function doFunc(n: number): string {
  for (let a = 0; a * a <= n; a++) {
    for (let b = 0; b * b <= n; b++) {
      if (a * a + b * b === n) {
        return "TRUE";
      }
    }
  }
  return "FALSE";
}
let result = doFunc(n);
console.log(result);
process.exit();
});

```

26. 服务器集群网络延迟

给定一个正整数数组表示某服务器集群内服务器的机位编号,请挑选一台服务器作为主服务器,使得集群网络延迟最小,并返回该最小值。

服务器之间的网络延迟,可以简单用相同机位编号之差的绝对值表示;服务器到母机的延迟是该主服务器与所有服务器的网络延迟之和。

解答要求

时间限制: 2000ms

内存限制: 200MB

输入

第一行整数 n ,代表数组元素的个数,取值范围: $[1,500]$ 。

第二行是 n 个数值,表示上述数组的元素(单个空格间隔),取值范围: $[1,10000]$ 。

输出

一个整数,表示集群网络延迟的最小值。

样例1

标准输入: 3 2 6 4

标准输出: 4

解释: 选择机位编号为4的服务器作为主服务器,集群网络延迟最小: $(|2-4| + |4-4| = 4)$



代码:

```
process.stdin.resume();
process.stdin.setEncoding('utf-8');
let input = "";
process.stdin.on('data', (data) => {
    input += data;
});
process.stdin.on('end', () => {
    let inputArray = input.split('\n');
    if (inputArray.length < 2) {
        console.error("输入数据不完整");
        return;
    }
    function doFunc() {
        let minDelay = Infinity;
        let n = parseInt(inputArray[0]);
        if (n < 1 || n > 5001) {
            console.error("输入的数组元素个数超出范围");
            return;
        }
        let numbers = inputArray[1].split(' ').map(Number);
        for (let num of numbers) {
            if (num < 1 || num > 10000) {
                console.error("输入的数组元素值超出范围");
                return;
            }
        }
        if (numbers.length !== n) {
            console.error("输入的数组元素个数与第一行声明的不符");
            return;
        }
        for (let i = 0; i < n; i++) {
            let currentDelay = 0;
            for (let j = 0; j < n; j++) {
                currentDelay += Math.abs(numbers[i] - numbers[j]);
            }
        }
    }
    doFunc();
    process.exit();
});
```

```
    }  
    if (currentDelay < minDelay) {  
        minDelay = currentDelay;  
    }  
}  
console.log(minDelay);  
}  
doFunc();  
process.exit();  
});
```

27. 查找新增字母

给定一个正整数数组表示某服务器集群内服务器的机位编号,请挑选一台服务器作为主服务器,使得集群网络延迟最小,并返回该最小值。

服务器之间的网络延迟,可以简单用相同机位编号之差的绝对值表示;服务器到母机的延迟是该主服务器与所有服务器的网络延迟之和。

解答要求

时间限制: 2000ms

内存限制: 200MB

输入

第一行整数 n ,代表数组元素的个数,取值范围: $[1,500]$ 。

第二行是 n 个数值,表示上述数组的元素(单个空格间隔),取值范围: $[1,10000]$ 。

输出

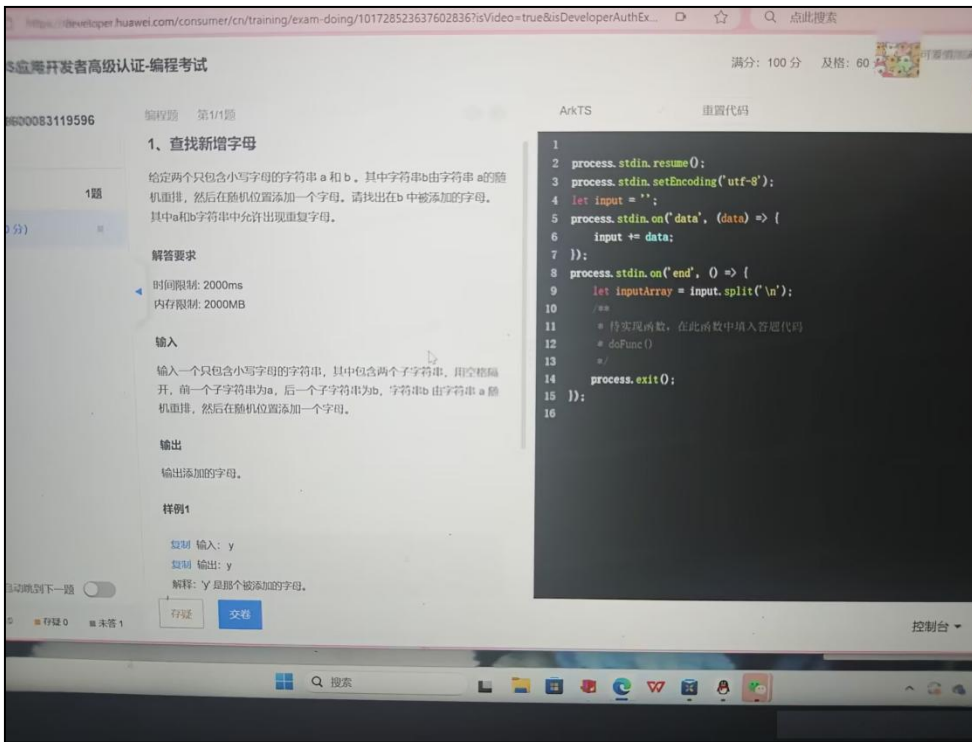
一个整数,表示集群网络延迟的最小值。

样例1

输入: 3 2 6 4

输出: 4

解释: 选择机位编号为4的服务器作为主服务器,集群网络延迟最小: $(|2-4| + |4-4| + |6-4|) = 4$ 。



代码：

```
process.stdin.resume();
process.stdin.setEncoding('utf-8');
let input = "";
process.stdin.on('data', (data) => {
    input += data;
});
process.stdin.on('end', () => {
    let inputArray = input.split('\n');

    function findAddedLetter(a: string, b: string): string {
        let charCountA: { [key: string]: number } = {};
        let charCountB: { [key: string]: number } = {};
        for (let char of a) {
            if (charCountA[char]) {
                charCountA[char]++;
            } else {
                charCountA[char] = 1;
            }
        }
        for (let char of b) {
            if (charCountB[char]) {
                charCountB[char]++;
            } else {
                charCountB[char] = 1;
            }
        }
        for (let char in charCountB) {
            if (!charCountA[char] || charCountB[char] > charCountA[char]) {
                return char;
            }
        }
        return "";
    }

    // Your code here
    process.exit(0);
});
```



```
function main() {
  let input = "y";
  let parts = input.split(" ");
  if (parts.length === 1) return console.log(input);
  if (parts.length === 2) {
    let addedLetter = findAddedLetter(parts[0], parts[1]);
    console.log("添加的字母是: " + addedLetter);
  } else {
    console.log("输入的格式不符合要求");
  }
}
main();
process.exit();
});
```

28. 寻找相同子串

给定两个字符串 **t** 和 **p**, 要求从 **t** 中找到一个和 **p** 相同的连续子串, 并输出该子串第一个字符在 **t** 中的下标。

解答要求

时间限制: 2000ms

内存限制: 2000kb

输入

输入文件包括两行, 分别表示字符串 **t** 和 **p**, 保证 **t** 的长度不小于 **p**, 且 **t** 的长度不超过 1000, **p** 的长度不超过 1000。

输出

若能找到, 则输出该子串第一个字符在 **t** 中的下标 (下标从左到右按照从 1 开始计数), 如果不能找到则输出 "No", 如果会有多个这样的子串, 则输出第一个字符下标最小的。

样例1

复制

输入:

AVERDXIVYERDIAN

RDXI

输出: 4

描述: 字符串中有 2 个相同的子串, 且第一个本应该的下标为 4, 故输出为 4。

样例2

输入:

abcde

fgh

输出: No

描述: 字符串中没有与 **p** 相同的子串, 故输出为 No。

1. 寻找相同子串

给定两个字符串 t 和 p，要求从 t 中找到一个和 p 相同的连续子串，并输出 p 子串第一个字符在 t 中的下标。

解法要求

时间限制: 2000ms
内存限制: 2000MB

输入

输入文件包括两行，分别表示字符串 t 和 p，保证 t 的长度不小于 p，且 t 的长度不超过 1000，p 的长度不超过 1000。

输出

如果能从 t 中找到一个和 p 相等的连续子串，则输出 p 子串第一个字符在 t 中的下标（下标从左到右依次为 1, 2, 3...）；如果不能则输出 "No"；如果会有多个这样的子串，则输出第一个字符下标最小的。

样例1

复制 输入: AVERDXIVYERDIAN
RDXI
复制 输出: 4
解释: 字符串 t 中有与 p 相同的连续子串，且第一个字符 R 在 t 中下标为 4，故输出为 4。

样例2

复制 输入: abode
cab
复制 输出: No
解释: 字符串 t 中没有与 p 相同的连续子串，故输出为 No。

```

1
2 process.stdin.resume();
3 process.stdin.setEncoding('utf-8');
4 let input = "";
5 process.stdin.on('data', (data) => {
6   input += data;
7 });
8 process.stdin.on('end', () => {
9   let inputArray = input.split('\n');
10  //
11  // 待实现函数，在此函数中填入答题代码
12  // doFunc()
13  //
14  process.exit(0);
15 });
16

```

代码:

```

process.stdin.resume();
process.stdin.setEncoding('utf-8');
let input = "";
process.stdin.on('data', (data) => {
  input += data;
});
process.stdin.on('end', () => {
  let inputArray = input.split('\n');
  const text = inputArray[0];
  const pattern = inputArray[1];
  function strStr(t: string, p: string): number|string {
    const tLength = t.length;
    const pLength = p.length;
    for (let i = 0; i <= tLength - pLength; i++) {
      let j;
      for (j = 0; j < pLength; j++) {
        if (t[i + j] !== p[j]) {
          break;
        }
      }
      if (j === pLength) {
        return i + 1;
      }
    }
    return "No";
  }
  const index = strStr(text, pattern);
  console.log(index);
  process.exit();
});

```

29. 找出公共字母

给你一个字符串数组words，找出并返回 $\text{length}(\text{words}[i]) * \text{length}(\text{words}[j])$ 的最大值，并且这两个单词不含有公共字母（公共字母：两个单词中相同的字母，如单词aecd和cbag的公共字母为ac）。如果不存在这样的两个单词，返回0。

解答要求

时间限制：2000ms

内存限制：2000MB

输入

输入只有一行，即一个字符串数组words，每个字符串仅包含小写字母。其中 $2 \leq \text{words.length} \leq 1000$, $1 \leq \text{words}[i].\text{length} \leq 1000$

输出

在一行输出一个整数，返回输出并返回 $\text{length}(\text{words}[i]) * \text{length}(\text{words}[j])$ 的最大值。

样例1

复制 输入：a ab abc d cd bcd abcd

复制 输出：4

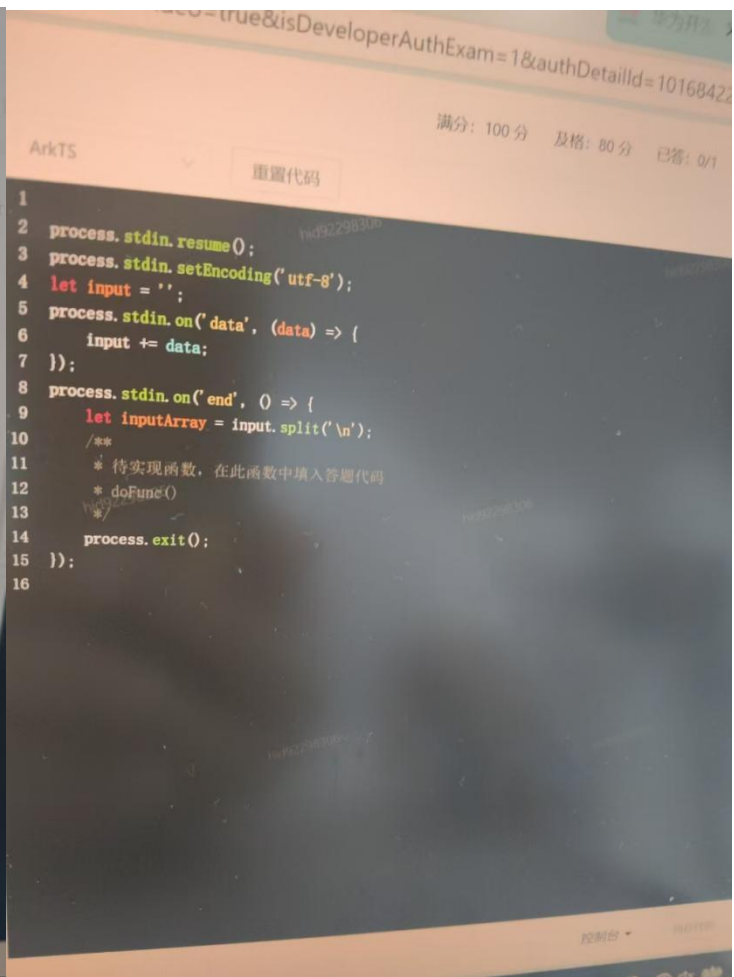
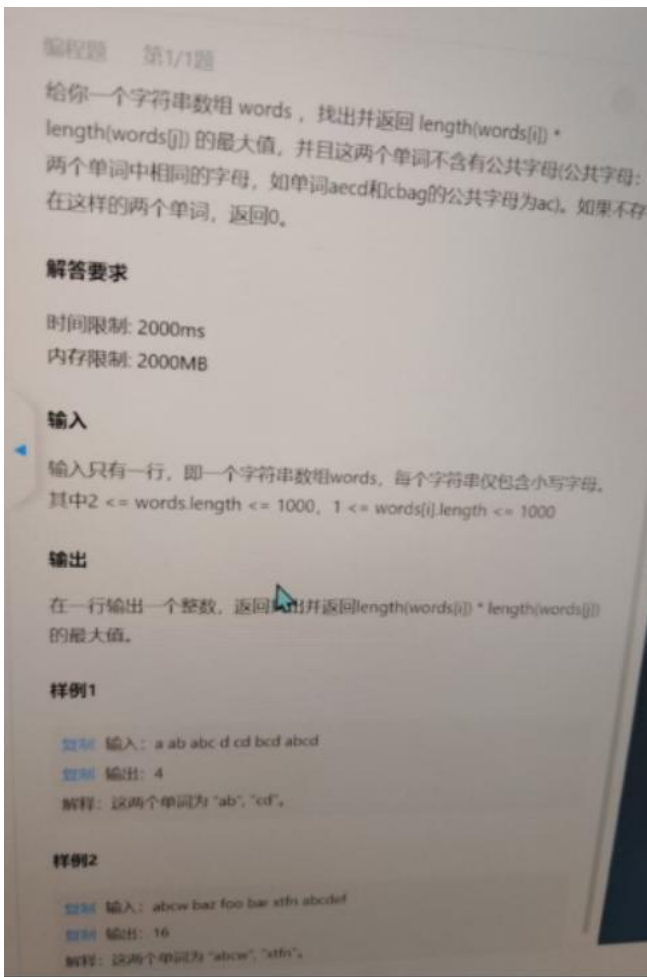
解释：这两个单词为“ab”，“cd”。

样例2

复制 输入：abcw baz foo bar xtn abcdef

复制输出：16

解释：这两个单词为“abcw”，“xtn”。



代码:

```
process.stdin.resume();
process.stdin.setEncoding('utf-8');
let input = '';
process.stdin.on('data', (data) => {
  input += data;
});
process.stdin.on('end', () => {
  let inputArray = input.split('\n');
  /**
   * 待实现函数，在此函数中填入答题代码
   * doFunc()
   */
  function doFunc() {
    // 1. 处理输入：把一行字符串按空格分割成单词数组，过滤空字符串
    // const inputLine = 'a ab abc d cd bcd abcd';
    // const inputLine = 'abcdw baz foo bar xfn abcdef';
    const inputLine = inputArray[0].trim();
    const words = inputLine.split(' ').filter(word => word.length > 0);

    // 2. 预处理：给每个单词生成「位掩码」，快速判断公共字母
    // 原理：每个小写字母对应1个二进制位，单词出现过该字母就把对应位置为1
    const masks: number[] = []; // 存每个单词的位掩码
    const wordLengths: number[] = []; // 存每个单词的长度
```

```

for (const word of words) {
  let mask = 0;
  for (const c of word) {
    // 字母转成对应的位 (a→0, b→1, ..., z→25)
    const bit = c.charCodeAt(0) - 'a'.charCodeAt(0);
    mask |= 1 << bit; // 把对应位设为1
  }
  masks.push(mask);
  wordLengths.push(word.length);
}

let maxProduct = 0;
const n = words.length;

// 3. 双重循环遍历所有两两单词，找无公共字母的最大长度乘积
for (let i = 0; i < n; i++) {
  for (let j = i + 1; j < n; j++) {
    // 两个掩码做「与运算」，结果为0说明没有公共字母
    if ((masks[i] & masks[j]) === 0) {
      const product = wordLengths[i] * wordLengths[j];
      if (product > maxProduct) {
        maxProduct = product;
      }
    }
  }
}

// 4. 输出结果（无符合条件时输出0）
console.log("maxProduct");
}

doFunc();
process.exit();
});

```

30. 和为 k 的子数组

给你一个整数数组 `nums` 和一个整数 `k`，请你统计并返回该数组中和为 `k` 的子数组的个数。子数组是数组中元素的连续非空序列，长度大于等于 1。

解答要求

时间限制: 2000ms

内存限制: 2000MB

输入

输入包含两行：

第一行为一个整数，代表k；

第二行为n个用单个空格隔开的整数，对应数组nums的元素。

约束条件：

$1 \leq n \leq 1000$

$-1000 \leq \text{nums}[i] \leq 1000$

$-1000 \leq k \leq 1000$

输出

输出数组中和为k的子数组的个数。

样例 1

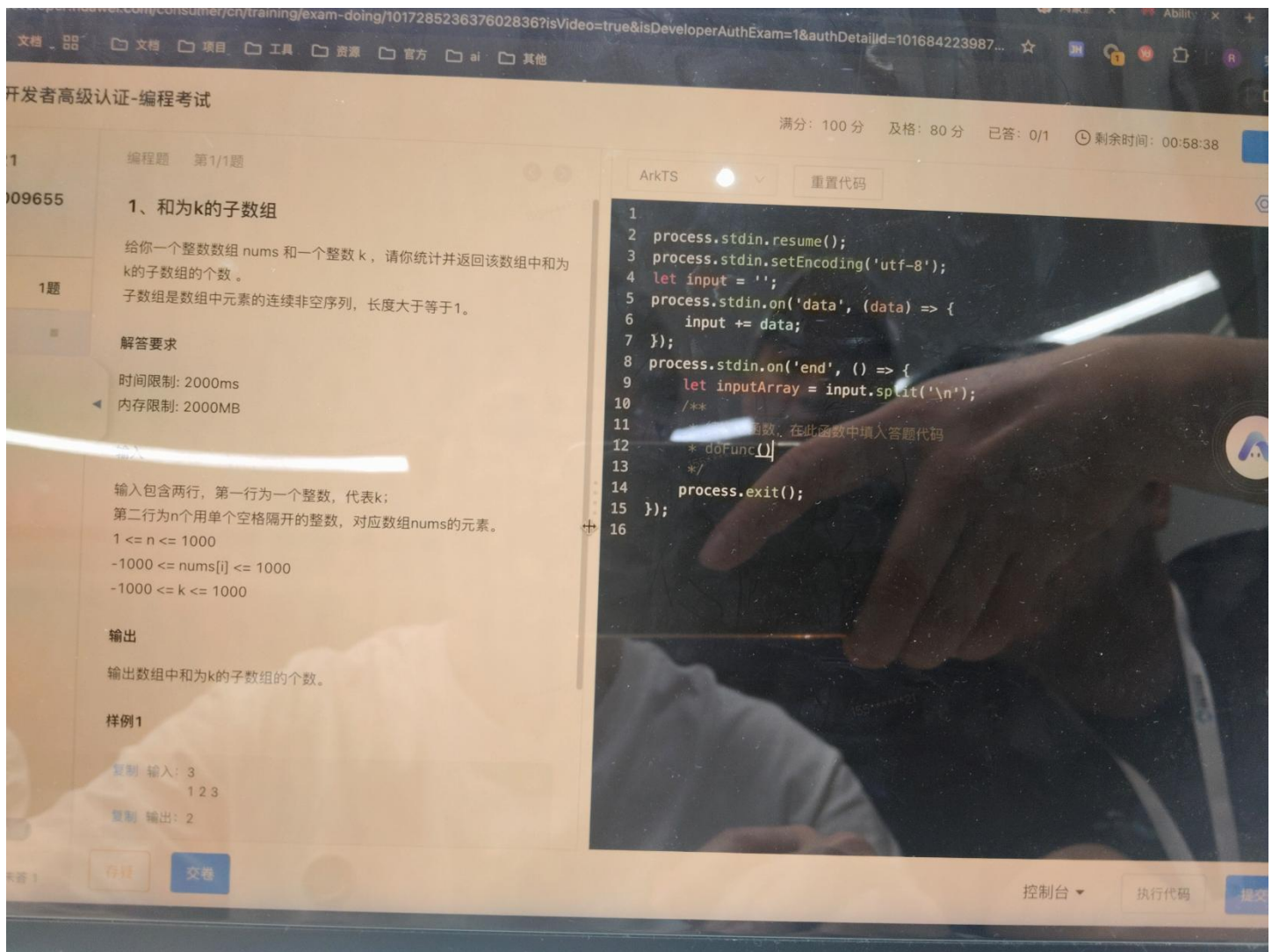
输入：

3

1 2 3

输出：

2



代码

```
process.stdin.resume();
process.stdin.setEncoding('utf-8');
let input = "";
process.stdin.on('data', (data) => {
    input += data;
});
process.stdin.on('end', () => {
    let inputArray = input.split('\n');
    /**
     * 待实现函数，在此函数中填入答题代码
     * doFunc()
     */
    function doFunc() {
        // 读取输入：k 和 nums 数组
        const k: number = parseInt(inputArray[0].trim(), 10);
        const nums: number[] = inputArray[1].trim().split(' ').map(Number);
        //const k: number = parseInt("2".trim(), 10);
        //const nums: number[] = "1 1 1".trim().split(' ').map(Number);
        let count = 0;
        // 暴力遍历所有子数组
        for (let i = 0; i < nums.length; i++) {
```

```
let sum = 0;
for (let j = i; j < nums.length; j++) {
    sum += nums[j];
    if (sum === k) {
        count++;
    }
}
}
}

console.log("",count);
}
doFunc();

process.exit();
});
```

31. 计算立方和

给定两个正整数 a 和 b ，要求计算出 a 到 b 之间所有整数的立方的和，即计算 $a^3+(a+1)^3+\dots+b^3$ 。

解答要求

时间限制: 2000ms

内存限制: 2000MB

输入

输入只有 1 行，包含 2 个整数 a, b ($1 \leq a \leq b \leq 200$)，用一个空格分开。

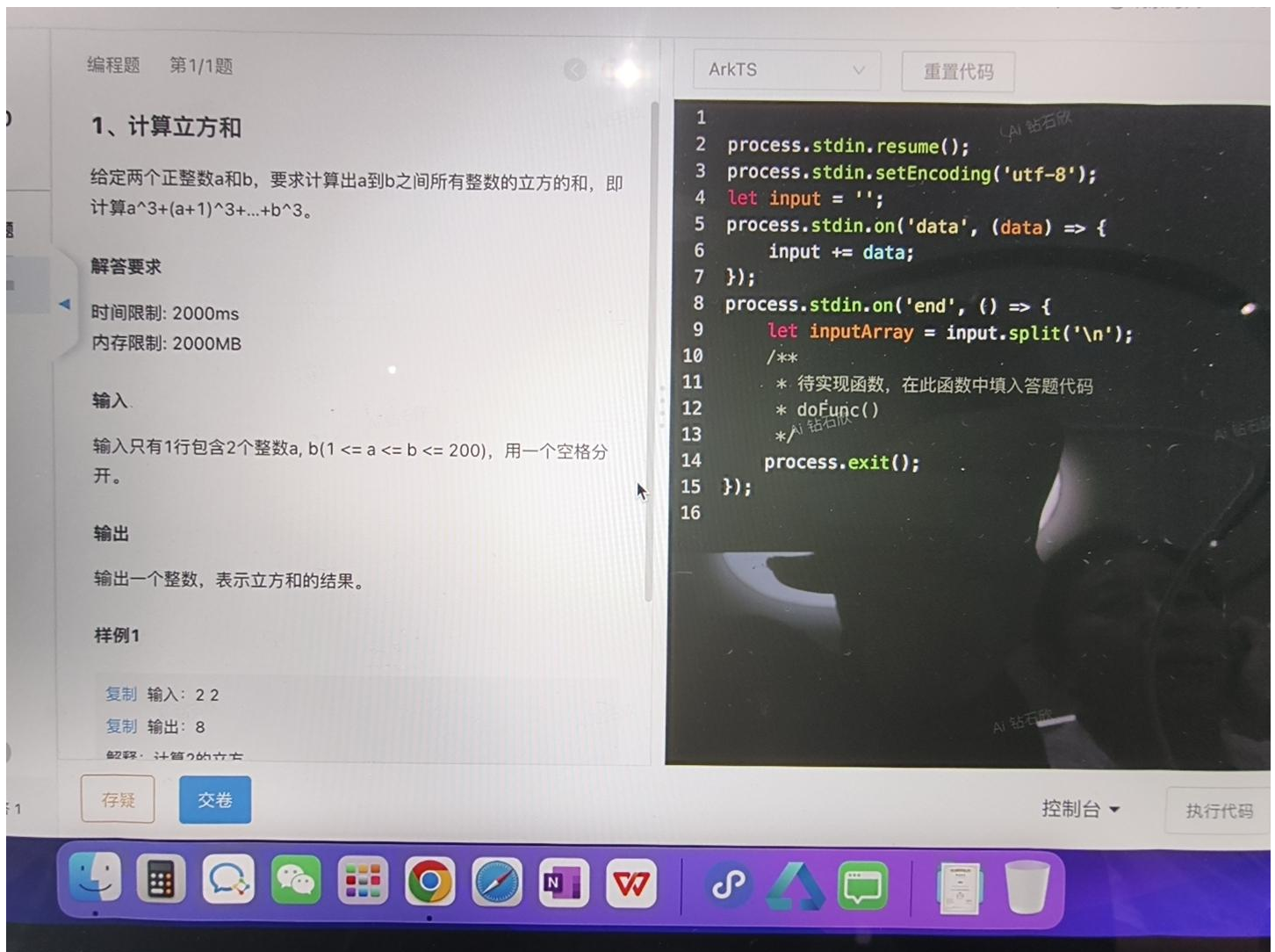
输出

输出一个整数，表示立方和的结果。

样例 1

输入: 2 2

输出: 8



【代码】

```
process.stdin.resume();
process.stdin.setEncoding('utf-8');
let input = "";
process.stdin.on('data', (data) => {
    input += data;
});
process.stdin.on('end', () => {
    let inputArray = input.split('\n');
    /**
     * 待实现函数，在此函数中填入答题代码
     * doFunc()
     */
    function doFunc() {
        // 读取输入的两个整数a和b
        // const line = inputArray[0].trim().split(' ');
        const line = "2 2".trim().split(' ');
        const a: number = parseInt(line[0], 10);
        const b: number = parseInt(line[1], 10);

        let sum = 0;
        // 计算a到b之间所有整数的立方和
```

```
for (let i = a; i <= b; i++) {  
  sum += i ** 3;  
}  
  
console.log("",sum);  
}  
doFunc();  
  
  process.exit();  
});
```